EXECUTIVE PRESENTATION · SAMPLE WORKBOOK

# From raw Excel to decisions you can audit

A reproducible local pipeline connects data quality, historical KPIs, a validated sales forecast and guarded next-week price proposals.

**Archive + raw** → **Clean star schema** → **Forecast + KPI** → **Pricing + dashboard**

**30**
products

**260**
observed weeks

**780**
latest future rows

**30**
reviewable price proposals

> Decision: use the dashboard for review, then test price changes in a controlled pilot before claiming uplift.

# Keep the raw record; make cleaning explicit

The loader accepts compatible Excel workbooks, archives exact bytes, and logs every repair or anomaly.

1 · PRESERVE

## Flexible intake

Headers within the first 10 rows; common aliases, case, punctuation and column order tolerated.

All parsed data rows kept in **raw_weekly**; extra cells in JSON; exact workbook bytes stored as a BLOB.

SHA-256 prevents duplicate loads of an unchanged file.

2 · VALIDATE / REPAIR

## Required-field gates

Positive integer week, nonempty product, nonnegative unit cost, positive unit price, nonnegative integer sales; opening stock cannot be negative.

Missing sales recovered only when **revenue ÷ price** implies whole units within one cent.

Equivalent product-week duplicates collapse; conflicting duplicates are excluded and logged.

3 · RECALCULATE / FLAG

## Warn without erasing

Revenue and gross profit recomputed from clean sales, price and cost; source mismatches logged.

Zero sales remain valid. Missing historical forecast stays NULL. Invalid optional fields become NULL with a reason.

Price ≤ cost, sales above opening stock, zero turnover with sales and week jumps are flagged for review.

**7,839**
raw rows

**7,800**
clean facts

**39**
equivalent duplicates

**16**
sales recovered

Sample: **0 rejected facts**. The 4,629 quality-issue records include warnings and information; they are not 4,629 rejected rows.

Source: assessment_etl/pipeline.py · README.md · ingestion_run/data_quality_issue

# Archive and clean tables · exact columns

SQLite stores money as integer cents in cleaned facts; raw input remains available for traceability.

### ingestion_run

1 rows

One archived workbook + status and row counts

`run_id, source_sha256, source_name, source_workbook, imported_at, loaded_at, status, error_message, raw_rows, accepted_rows, rejected_rows, duplicate_rows`

### raw_weekly

7,839 rows

Every parsed Excel row, including duplicate or invalid rows

`raw_id, run_id, sheet_name, excel_row, week, product, sales_units, cost_of_goods_sold_cogs, inventory_level, demand_forecast, price_per_unit, gross_profit, operating_profit, revenue, inventory_turnover_ratio, gross_profit_margin, revenue_growth, extra_cells_json`

### data_quality_issue

4,629 rows

One coded issue; warnings are not rejections

`issue_id, run_id, raw_id, severity, issue_code, detail, created_at`

### dim_product

30 rows

One normalized product

`product_id, product_key, product_name`

### dim_week

286 rows

One relative week, including future forecast weeks

`week_id, relative_year, week_in_year`

### fact_weekly_product

7,800 rows

One cleaned product × observed week

`product_id, week_id, sales_units, unit_cost_cents, opening_inventory_units, historical_forecast_units, unit_price_cents, revenue_cents, gross_profit_cents, operating_profit_cents, turnover_ratio_source, sales_recovered, source_run_id, source_raw_id, loaded_at`

Rows shown are from this local database. Table names and every column above come from **PRAGMA table_info**; source workbook has no calendar dates.

# Forecast and pricing tables · exact columns

Historical actuals, model runs and proposed prices have separate grains and separate version histories.

### forecast_run

2 rows

Versioned model selection and evaluation metadata

```
forecast_run_id, run_key, created_at, input_max_week, horizon_weeks,
input_sha256, feature_version, selected_method, selection_rule,
selected_parameters_json, development_metrics_json, holdout_metrics_json,
training_rows
```

### forecast_backtest

43,680 rows

One method × fold × product × target week prediction

```
forecast_run_id, method, fold_number, origin_week, product_id, target_week,
actual_units, predicted_units
```

### fact_demand_forecast

1,560 rows

One run × product × future week forecast

```
forecast_run_id, product_id, week_id, predicted_units,
lower_error_band_units, upper_error_band_units
```

### pricing_run

1 rows

Versioned price policy, inputs and forecast reference

```
pricing_run_id, run_key, created_at, forecast_run_id, base_week,
target_week, policy_version, policy_json, input_sha256
```

### fact_price_recommendation

30 rows

One run × product × target week proposal

```
pricing_run_id, product_id, target_week, current_price_cents,
unit_cost_cents, minimum_price_cents, recommended_price_cents,
forecast_units, recent_mean_units, forecast_vs_recent_ratio, demand_signal,
decision, reason_code, opening_stock_cover_days_proxy, inventory_warning,
break_even_units
```

### Five read-only views

Calculated presentation layer; no extra stored source rows

```
vw_weekly_kpi · product/week;
vw_kpi_product_week · detailed actual KPIs;
vw_kpi_portfolio_week · portfolio totals;
vw_latest_demand_forecast · latest 780 rows;
vw_latest_price_recommendation · latest 30 proposals.
```

Why 1,560 forecast rows? Two 780-row runs remain stored; **vw_latest_demand_forecast** exposes only the newest run.

Source: sql/schema.sql · data/assessment.sqlite

04 · FORECASTING APPROACH

# Ten candidates, one time-split protocol

Direct 26-week forecasts use product, seasonal position, historical sales and prices known at each origin.

### 2 TRANSPARENT BASELINES

- `seasonal_52`
  Same week 52 weeks earlier
- `recent_13_mean`
  Mean of the latest 13 weeks

Useful checks against an expensive model and simple fallbacks if accuracy is close.

### 5 GRADIENT BOOSTING LOSSES

- `gbdt_mae` · absolute error
- `gbdt_rmse` · squared error
- `gbdt_poisson` · Poisson
- `gbdt_q25` · 25th quantile
- `gbdt_q75` · 75th quantile

### 3 RANDOM FOREST SPLIT CRITERIA

- `forest_mae` · absolute error
- `forest_rmse` · squared error ✓ **selected**
- `forest_poisson` · Poisson

Forest criteria guide tree splits; they are not the same object as boosting's global loss.

**605.7**
selected MAE · units

**814.0**
selected RMSE · units

**47.51%**
selected WAPE

**5.4%**
MAE below best boosting

Select by pooled MAE on the first three 26-week windows. The fourth was inspected earlier, so it is not a fresh holdout. MAPE excludes 431 zero-sales rows in development.

Source: reports/forecast_summary.md · reports/forecast_evaluation.json

05 · KPI INSIGHTS AND DASHBOARD

# Read the business picture locally

Recent 13 observed weeks (248–260) versus the previous complete 13-week period; source currency is unspecified.

| 16.04m | +3.09% | 21.49% | 489,871 |
|---|---|---|---|
| revenue | revenue change | gross profit margin | units sold |

## OPERATIONAL SIGNALS

- Week 260 revenue is **15.43% below** week 259.
- 8 products report zero opening stock; 9 sold more than opening stock.
- These are diagnostics, not confirmed stockouts: in-week receipts are unknown.
- Portfolio margin = total gross profit ÷ total revenue.

## STREAMLIT WALKTHROUGH

### Five tabs, one product filter

**Overview** · **Demand forecast** · **Inventory diagnostics** · **Pricing rules** · **Data and method**

Filter by product; inspect historical trends, latest forecast, warning flags and proposal reasons; export the selected rows as CSV.

Run locally: **python -m streamlit run dashboard/app.py** using the project venv. Formal DSI, DSO and default-risk scores are unavailable from this workbook.

Source: reports/kpi_summary.json · reports/kpi_definitions.md · dashboard/app.py

06 · DYNAMIC PRICING

# Rules propose, people approve

Compare next-week forecast with the previous 13-week sales mean; propose one bounded move for week 261.

**Forecast ≥ 1.20 × recent mean**

## +3% price

High sales signal, subject to policy guardrails.

**Forecast ≤ 0.80 × recent mean**

## −3% price

Low sales signal, subject to policy guardrails.

**Otherwise**

## Hold

Missing/zero reference sales normally hold; margin floor may override.

### Hard constraints

- Minimum **15% gross unit margin**.
- Maximum **5% change** from current price.
- A conflicting floor and cap yields **review**, not an invented price.

### Current sample outcome

- **11 increases**, 19 holds, 0 decreases or reviews.
- **15 inventory warnings** ask for a human stock check; they do not drive the price rule.
- Every proposal stores its reason and policy run ID.

Break-even units = forecast units × current unit margin ÷ proposed unit margin, rounded up. This is a gross-profit threshold, not forecast sales at the new price.

Source: reports/pricing_strategy.md · assessment_etl/pricing.py

07 · ROI SENSITIVITY AND NEXT DECISION

# Economics require an observed price response

Illustrative one-week sensitivity for the 11 proposed increases. New-price sales volume is unknown.

## Against current-price gross profit

| VOLUME AT NEW PRICES | Δ GROSS PROFIT | ROI* |
|---|---|---|
| 0% fewer units | +12,579 | +152% |
| 5% fewer units | +7,286 | +46% |
| 10% fewer units | +1,993 | -60% |
| 15% fewer units | -3,300 | -166% |

* ROI = (incremental gross profit − assumed weekly cost) ÷ assumed weekly cost. Cost assumption: 5,000 source-currency units.

CONDITIONAL THRESHOLD

## ≈ 7.2% fewer units

At this loss, incremental gross profit just covers the assumed 5,000 weekly cost. This depends on a volume response we have not measured.

## Before a live rollout

- Confirm current inventory, unit costs and business-approved guardrails.
- Run a controlled price pilot and measure incremental gross profit and execution cost.

The ROI numbers are scenario arithmetic, **not validated uplift**. Sales are a proxy for demand; no causal elasticity, in-week receipts, real calendar dates, formal DSI/DSO or customer credit data are available.

Source: data/pricing_report.json · reports/pricing_strategy.md · ROI scenario uses latest price/cost and forecast units